Contender 14: Self-Indexed Diagonal Operator Reflection

A constructive large-number proposal for BusyLean

Research proposal prepared for Vladimir Reshetnikov

October 13, 2026

Abstract

We propose a new closed computable natural number for the BusyLean contest, targeting repository commit `1ee50c7d97bbed57a60709f661453c3e213f4db3`, whose incumbent is Contender 13. Contender 13 performs one reflection step over a predicative "dynamic-kind" extension of System F_ω, with Contender 12's family as the unary oracle. The proposed Contender 14 makes three independent upgrades. First, it lifts dynamic-kind reflection from a unary oracle to a packed countable oracle family, while retaining an explicit prefix maximum that gives direct strict inequalities without any false monotonicity assumption. Second, it turns this family reflector into a progressive transformer and iterates the repository's operator-stage construction through rank-dependent autonomous schedules. Third, it diagonalizes the rank: at input n, the number of operator-stage levels is

$$\mathsf{family13}(n) = F_{13}(F_{13}(n)),$$

so at input 42 the tower rank is definitionally the entire previous contender. The executable definition nevertheless does not mention the constant `contender_13`; it uses the predecessor's construction, as the contest rules permit.

The supplied Lean development states and proves, at source level, that every finite tower rank is progressive, every lower rank is strictly below every higher rank along explicit embedded branches, every enumerated dynamic-kind program at depth n is below $F_{14}(n)$, $\mathsf{family13}(n) < F_{14}(n)$ for every n, and finally

$$\mathsf{contender_13} < \mathsf{contender_14}.$$

All new definitions are transparent and structurally recursive, and the archive includes an integration patch, static audit, and reproduction notes. The current execution environment did not contain Lean 4.34.0 or the repository checkout, so kernel compilation and timing remain mandatory integration checks; this limitation is recorded explicitly rather than hidden.

Contents

1 Executive statement of the proposal **3**

2 Repository baseline and contest constraints **3**

2.1 Baseline 3

2.2 Relevant rules 4

3 Why a new layer is available above Contender 13 **4**

4 Family-aware dynamic-kind reflection **5**
4.1 Definition . . . 5
4.2 Direct inequalities . . . 6
4.3 Why no monotonicity is used . . . 6

5 The base transformer **6**

6 Rank-dependent diagonal schedules **7**
6.1 Existing autonomous notation machinery . . . 7
6.2 The schedule . . . 7

7 The diagonal operator tower **8**
7.1 Definition . . . 8
7.2 Progressivity of every rank . . . 8
7.3 Every successor rank is strictly larger . . . 8

8 Self-indexing by the previous construction **9**
8.1 The predecessor family . . . 9
8.2 The self-indexed tower and final family . . . 9

9 Formal comparison with Contender 13 **10**
9.1 Pointwise family comparison . . . 10
9.2 Input growth . . . 10
9.3 Domination of the reflected programs . . . 10
9.4 The required closed comparison . . . 11

10 Why this is a significant rather than cosmetic increase **11**
10.1 Why no exact fast-growing calibration is claimed . . . 11

11 Computability and consistency discipline **12**
11.1 Totality of the new definitions . . . 12
11.2 No new normalization theorem is needed . . . 12
11.3 Expected axiom closure . . . 12

12 No-laziness analysis **13**

13 Alternatives considered and rejected **13**
13.1 A single second application of `F13` . . . 13
13.2 One operator stage over the dynamic reflector . . . 13
13.3 Assuming oracle monotonicity . . . 13
13.4 A self-containing dynamic universe . . . 14
13.5 Using the predecessor constant as the tower height . . . 14

14 Integration and validation protocol **14**
14.1 What has been validated in this archive . . . 14
14.2 What has not been validated here . . . 15

15 Further research questions **15**
15.1 Formal questions directly suggested by Contender 14 15
15.2 Stronger language directions 15
15.3 Calibration and metamathematics 16

16 Conclusion **16**

A Theorem dependency map **17**

B Lean definition module **17**

C Lean comparison module **19**

1 Executive statement of the proposal

The proposal consists of the following definitions.

$$
\begin{aligned}
R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}(fs, n) &= n + 1 + \max\left(\max_{i \le n} fs(i, n),\ \max_{t \in \mathrm{CTm}_{\le n}} \mathrm{value}_{\mathsf{F}_{\mathsf{dyn}}}(\mathrm{pack}(fs), n, t)\right), \\
T_0 &= \text{the transformer induced by } R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}, \\
T_{k+1} &= \mathsf{OpStage}(T_k, \mathsf{Schedule}(k)), \\
\mathsf{family13}(n) &= F_{13}(F_{13}(n)), \\
F_{14}(n) &= T_{\mathsf{family13}(n)}(\text{constant family } \mathsf{family13})(n, n), \\
C_{14} &= F_{14}(F_{14}(42)).
\end{aligned}
$$

The rank-dependent schedule $\mathsf{Schedule}(k)$ has one deliberately simple branch and countably many very large branches:

$$
\mathsf{Schedule}(k) = \mathrm{node}\left(j \mapsto \begin{cases} \mathrm{leaf}, & j = 0, \\ \mathrm{realize}(\mathrm{hyper}_{k+r+1}(\mathrm{encode}(\mathsf{tree12}))), & j = r + 1. \end{cases}\right).
$$

The leaf is not cosmetic. It is a proof witness: it embeds one ordinary autonomous lift inside each new operator stage, allowing a direct strict comparison even though reflection is not monotone in its oracle. The other branches make the schedule genuinely diagonal and unbounded in the explicit higher-derivative notation already present in the repository.

The final Lean line is intentionally simple:

```
def contender_14 : Nat := F14 (F14 42)
```

The size comes from the construction of `F14`, not from an enormous literal or an unproved external oracle.

2 Repository baseline and contest constraints

2.1 Baseline

This work targets the BusyLean main branch at the following commit:

`1ee50c7d97bbed57a60709f661453c3e213f4db3`.

At that commit the repository uses Lean 4.34.0 and declares Contender 13 as the current entry:

```
def family12 (n : Nat) : Nat := F12 (F12 n)
def F13 (n : Nat) : Nat := Fd.reflectFd family12 n
def contender_13 : Nat := F13 (F13 42)
```

The repository already contains four ingredients needed here:

1. `Fd.reflectFd`, a total reflector for the predicative dynamic-kind language;
2. `packFamily`, an executable coding of a countable family of unary functions as one unary oracle;
3. `operatorStage`, a generic closure operation on family transformers;
4. `Autonomous.hyper`, `encode`, and `realize`, which build transparent countably branching schedules above an existing tree.

The proposal composes these existing certified mechanisms rather than adding a new unverified interpreter or a new normalization theorem.

2.2 Relevant rules

The construction is designed around the repository's stated rules:

- the number is a closed, constructive `Nat`;
- the previous contender must be beaten by a formal theorem;
- no `noncomputable`, `partial`, `unsafe`, custom axiom, `opaque` stand-in, `extern`, `implemented_by`, `native_decide`, or `sorry` is used;
- only the inherited permitted axioms `propext` and `Quot.sound` may occur in proof closures;
- the previous contender constant must not be used as a cheap numerical parameter, although its underlying family construction may be used as an oracle, seed, or component of a qualitatively stronger mechanism;
- definitions and proofs must satisfy the repository's 15-second and 60-second phase budgets on the designated toolchain.

The last item is an empirical requirement. Source inspection can establish structural computability and dependency discipline, but only an actual clean Lean build can establish elaboration, kernel acceptance, axiom reports, and wall-clock timing.

3 Why a new layer is available above Contender 13

Contender 13 is a major language-strength jump. Its object language contains a predicative dynamic kind interpreted semantically as a dependent sum of operators at all dynamic-free kinds. It can quantify over two levels of kind variables and climb finite extensions above the dynamic kind at run time. Nevertheless, the numerical wrapper around this language is still a direct unary reflection:

$$F_{13}(n) = \mathrm{reflectFd}(\mathsf{family12}, n).$$

This leaves two orthogonal amplification directions.

Family access. A unary oracle gives one function $\mathbb{N} \to \mathbb{N}$. A forest or operator stage needs a countable family $fs : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ so an object-language program can select a child or coordinate dynamically. The earlier Contenders 9 and 11 solve this by packing (i, m) into one natural input. Applying the same idea to the stronger dynamic-kind language yields a family-aware dynamic reflector.

Operator closure. Contender 12 already showed that a progressive reflector can be lifted from a single family transformation to an operator-stage closure along large schedules. Contender 13 strengthened the object language but returned to one direct reflection step. The most economical next move is therefore not a new syntax of doubtful consistency, but to feed the stronger reflector into the existing generic operator machinery and then diagonalize the number of such closures.

The construction is consequently modular:

$$\boxed{\text{stronger object language}} \quad + \quad \boxed{\text{family reflection}}$$

$$\boxed{\text{operator-stage tower}} \quad + \quad \boxed{\text{self-indexed rank}}.$$

4 Family-aware dynamic-kind reflection

4.1 Definition

Let $\mathsf{Fam} = \mathbb{N} \to \mathbb{N} \to \mathbb{N}$. The repository's `packFamily` maps $fs \in \mathsf{Fam}$ to a unary oracle by decoding an input into a pair (i, m) and returning $fs(i, m)$. We define

Definition 4.1 (Family-aware dynamic reflector)**.** For $fs \in \mathsf{Fam}$ and $n \in \mathbb{N}$, let

$$\begin{aligned} P(fs, n) &= \max_{i \le n} fs(i, n), \\ Q(fs, n) &= \max_{t \in \mathrm{CTm}_{\le n}} \mathrm{fvalue}_{\mathsf{F}_{\mathsf{dyn}}}(\mathrm{packFamily}(fs), n, t), \\ R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}(fs, n) &= n + 1 + \max(P(fs, n), Q(fs, n)). \end{aligned}$$

The exact Lean definition is:

```
def reflect14 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (Fd.fvalue (packFamily fs) n) (Fd.ctmsUpTo n))
```

There are two logically distinct maxima.

- The prefix maximum provides a direct, syntax-independent inequality for every visible family coordinate.
- The program maximum lets every accepted dynamic-kind program of bounded syntax/fuel query the packed family oracle and be exceeded.

Neither is redundant. Packing gives expressive access to the whole family, but `Fd.reflectFd` by itself only visibly exceeds the unary oracle at its current input. Conversely, the prefix maximum alone would dominate coordinates but would not expose the family to object-language computation.

4.2 Direct inequalities

The arithmetic proof follows the pattern already used by the repository's other reflectors.

Lemma 4.2 (Arithmetic reflection bound)**.** *If*

$$m \le \max(P(fs,n), Q(fs,n)),$$

then $m < R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}(fs,n)$.

Proof. The defining right-hand side is $n+1+M$ with $M = \max(P,Q)$. From $m \le M$ and $M < M+1 \le n+1+M$, transitivity gives $m < R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}(fs,n)$. □

Corollary 4.3 (Visible-family domination)**.** *For every* $i \le n$,

$$fs(i,n) < R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}(fs,n).$$

Proof. The finite prefix maximum contains $fs(i,n)$, so $fs(i,n) \le P(fs,n) \le \max(P,Q)$; apply the arithmetic bound. □

Corollary 4.4 (Program domination)**.** *For every* $t \in \mathrm{CTm}_{\le n}$,

$$\mathrm{fvalue}_{\mathsf{F}_{\mathsf{dyn}}}(\mathrm{packFamily}(fs), n, t) < R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}(fs,n).$$

Proof. The finite maximum $Q(fs,n)$ contains the program value; apply the arithmetic bound. □

4.3 Why no monotonicity is used

A recurring trap in large-number reflection arguments is the claim that making an oracle pointwise larger can only make every program value larger. That is false for ordinary programmable oracles: a program may compare, branch, or return a small result when an oracle value crosses a threshold. The BusyLean repository correctly avoids this assumption, and Contender 14 does the same. Every strict comparison follows an explicit occurrence in a finite maximum or an explicit branch in a tree. This discipline is essential because the later operator construction nests oracle-dependent interpreters many times.

5 The base transformer

A transformer maps one oracle family to another:

$$\mathsf{Transformer} = \mathsf{Fam} \to \mathsf{Fam}.$$

The family-aware reflector becomes

$$T_0(fs)(o,n) = R^{\mathrm{fam}}_{\mathsf{F}_{\mathsf{dyn}}}(fs,n),$$

where the output coordinate o is deliberately ignored.

```
def fdFamilyTransformer : Transformer := fun fs _ n => reflect14 fs n
```

The generic operator library uses the following invariant.

Definition 5.1 (Progressive transformer)**.** A transformer T is progressive when

$$\forall fs, i, o, n, \quad i \le n \Longrightarrow fs(i, n) < T(fs)(o, n).$$

Theorem 5.2 (Base progressivity)**.** *`fdFamilyTransformer` is progressive.*

Proof. For visible $i \le n$, the desired inequality is exactly visible-family domination. The arbitrary output coordinate disappears because the base transformer ignores it. □

This one-line invariant is what makes the rest of the construction reusable. No semantic property of the dynamic-kind language is needed beyond the already certified total evaluator used inside `Fd.fvalue`.

6 Rank-dependent diagonal schedules

6.1 Existing autonomous notation machinery

The repository contains a transparent tree syntax with zero, successor, and countable limit nodes, together with operations suggestive of ordinal addition, multiplication, ω-powers, derivatives, a finite critical hierarchy, a Gamma-like diagonal closure, and higher derivatives. It deliberately treats these as tree syntax rather than asserting quotient-level ordinal identities. The functions `encode` and `realize` convert between this syntax and the W-tree schedules used by reflection.

Contender 12 defines `tree12` by applying 42 higher derivatives above the language-defined index. We use `tree12` as a baseline and then build a schedule family rather than one fixed tree.

6.2 The schedule

For rank k, define

$$\mathsf{Schedule}(k) = \mathrm{node}(b_k)$$

with

$$b_k(j) = \begin{cases} \mathrm{leaf}, & j = 0, \\ \mathrm{realize}(\mathrm{hyper}_{k+r+1}(\mathrm{encode}(\mathsf{tree12}))), & j = r + 1. \end{cases}$$

The Lean code is:

```
def schedule14 (k : Nat) : WTree Nat :=
  .node (fun j =>
    match j with
    | 0 => .leaf
    | r + 1 =>
        Autonomous.realize
          (Autonomous.hyper (k + r + 1) (Autonomous.encode tree12)))
```

The two branch classes have different jobs.

Branch zero: a formal witness. The leaf ensures that the operator stage based on `schedule14 k` contains an explicit visible leaf-stage child. By definition, this child is `liftTransformer base`. The generic theorem `transformer_lt_lift_at` then gives a strict path

$$T(fs)(i, n) < \mathrm{lift}(T)(fs)(0, n) = \mathrm{OpStage}(T, \mathrm{leaf})(fs)(0, n) < \mathrm{OpStage}(T, \mathsf{Schedule}(k))(fs)(o, n).$$

This is a direct structural embedding, not a monotonicity argument.

Positive branches: diagonal growth. Branch $r+1$ uses a higher-derivative level $k+r+1$. Thus each fixed rank k contains countably many schedules of unbounded explicit hyper-level, and the minimum positive level also increases with k. No exact proof-theoretic ordinal is claimed. The formal role of these branches is to make the schedule genuinely richer; the strict inequality proof needs only branch zero.

7 The diagonal operator tower

7.1 Definition

The generic `operatorStage` accepts a base transformer and a W-tree and returns a new transformer. We iterate it with a schedule that changes at every rank:

```
def diagonalOperatorTower (schedule : Nat → WTree Nat)
    (base : Transformer) : Nat → Transformer
  | 0 => base
  | k + 1 =>
      operatorStage (diagonalOperatorTower schedule base k) (schedule k)
```

In mathematical notation,

$$T_0 = T_{\text{base}}, \qquad T_{k+1} = \mathsf{OpStage}(T_k, \mathsf{Schedule}(k)).$$

This is a genuine iteration of an operator closure: the entire transformer at rank k becomes the base transformer at rank $k+1$.

7.2 Progressivity of every rank

Theorem 7.1 (Tower progressivity)**.** *If T_0 is progressive, then every T_k is progressive.*

Proof. Induct on k. The zero case is the hypothesis. At a successor, the repository theorem `operatorStage_progressive` says that applying an operator stage to any progressive base transformer yields a progressive transformer, for every schedule tree. Instantiate it with T_k and $\mathsf{Schedule}(k)$. □

The Lean proof is correspondingly short:

```
theorem diagonalOperatorTower_progressive
    (schedule : Nat → WTree Nat) (base : Transformer)
    (hbase : Progressive base) (k : Nat) :
    Progressive (diagonalOperatorTower schedule base k) := by
  induction k with
  | zero => simpa [diagonalOperatorTower] using hbase
  | succ k ih =>
      simpa [diagonalOperatorTower] using
        (operatorStage_progressive
          (diagonalOperatorTower schedule base k) ih (schedule k))
```

7.3 Every successor rank is strictly larger

Progressivity alone says that each rank exceeds its seed family. The canonical leaf branch proves more: rank $k+1$ directly and strictly embeds rank k.

Theorem 7.2 (Strict rank step)**.** *For every seed family, every k, every $i \le n$, and every output coordinate o,*

$$T_k(\text{seed})(i, n) < T_{k+1}(\text{seed})(o, n).$$

Proof. By tower progressivity, T_k is progressive. The explicit path through branch zero of $\mathsf{Schedule}(k)$ gives

$$T_k(\text{seed})(i, n) < \text{lift}(T_k)(\text{seed})(0, n) < \mathsf{OpStage}(T_k, \mathsf{Schedule}(k))(\text{seed})(o, n).$$

The rightmost term is definitionally $T_{k+1}(\text{seed})(o, n)$. □

The successor theorem admits a finite transitive closure.

Theorem 7.3 (Strict comparison of arbitrary ranks)**.** *For every seed family, $a < b$, $i \le n$, and output coordinate o,*

$$T_a(\text{seed})(i, n) < T_b(\text{seed})(o, n).$$

Proof. Induct on b. The zero case is impossible. For $b = c + 1$, either $a = c$, in which case the successor theorem applies directly, or $a < c$. In the latter case, use the induction hypothesis to reach coordinate zero of rank c, then use one successor step from rank c to rank $c + 1$. Every intermediate coordinate is explicitly visible, so no monotonicity premise is introduced. □

This is the formal large-margin certificate. The final construction does not merely wrap Contender 13 once: its rank parameter at input 42 is the previous contender, and the source proves a strict comparison between every pair of ranks below that height.

8 Self-indexing by the previous construction

8.1 The predecessor family

Following the repository's convention, expose the full Contender 13 construction as a function of its input:

```
def family13 (n : Nat) : Nat := F13 (F13 n)
```

At input 42 this is definitionally the previous contender:

$$\mathsf{family13}(42) = F_{13}(F_{13}(42)) = C_{13}.$$

The new executable definition refers to `F13` and `family13`, not to the closed constant `contender_13`. This matters for the no-laziness audit: the prior construction is used as a seed and meta-rank for a stronger closure mechanism, while the old answer itself is absent from the dependency closure of the new answer.

8.2 The self-indexed tower and final family

```
def tower14 (n : Nat) : Transformer :=
  diagonalOperatorTower schedule14 fdFamilyTransformer (family13 n)

def F14 (n : Nat) : Nat :=
  tower14 n (fun _ => family13) n n
```

Three occurrences of n serve different roles.

1. $\mathsf{family13}(n)$ chooses the tower rank.
2. The output coordinate is n; every progressive transformer dominates all seed coordinates $i \le n$ there.
3. The numerical input is n, setting the syntax/fuel bound used whenever the dynamic-kind reflector is reached.

The seed family is constant in its first coordinate: $\mathrm{seed}(i, m) = \mathsf{family13}(m)$. Choosing visible coordinate $i = 0$ therefore exposes exactly $\mathsf{family13}(n)$ in the progressivity theorem.

9 Formal comparison with Contender 13

9.1 Pointwise family comparison

Theorem 9.1 (`family13_lt_F14`)**.** *For every n,*

$$\mathsf{family13}(n) < F_{14}(n).$$

Proof. The selected tower rank is progressive. Apply progressivity to the constant seed family, visible coordinate $i = 0$, output coordinate $o = n$, and numerical input n. Since $0 \le n$, the left side is $\mathrm{seed}(0, n) = \mathsf{family13}(n)$, while the right side unfolds to $F_{14}(n)$. □

The proof does not need to unfold the astronomical rank $\mathsf{family13}(n)$. It uses the theorem that every finite rank is progressive, instantiated symbolically at that rank.

9.2 Input growth

The repository proves $n < \mathrm{reflectFd}(f, n)$ for every oracle f. Therefore

$$n < F_{13}(n) < F_{13}(F_{13}(n)) = \mathsf{family13}(n) < F_{14}(n).$$

This yields:

Theorem 9.2 (`input_lt_F14`)**.** *For every n, $n < F_{14}(n)$.*

9.3 Domination of the reflected programs

The rank-zero transformer is already the family-aware dynamic-kind reflector. Since $0 < \mathsf{family13}(n)$, the arbitrary-rank theorem places that entire rank below the selected final rank. Combining this path with `fvalue_lt_reflect14` gives the following concrete language-level certificate.

Theorem 9.3 (`fvalue_lt_F14`)**.** *For every $t \in \mathrm{CTm}_{\le n}$,*

$$\mathrm{value}_{\mathsf{F}_{\mathsf{dyn}}}(\mathrm{pack}(i, m \mapsto \mathsf{family13}(m)), n, t) < F_{14}(n).$$

Thus the comparison is not based only on the seed value. Every bounded program in the incumbent's strongest certified object language, evaluated against the packed predecessor family, is strictly below the new family at the same input. The theorem is deliberately limited to the canonical rank-zero reflection node; domination of programs at every internal node is left as a stronger path-indexing problem.

9.4 The required closed comparison

Define

$$C_{14} = F_{14}(F_{14}(42)).$$

Then

$$\begin{aligned} C_{13} &= \mathsf{family13}(42) \\ &< F_{14}(42) && \text{by pointwise family comparison} \\ &< F_{14}(F_{14}(42)) && \text{by input growth} \\ &= C_{14}. \end{aligned}$$

The Lean theorem is:

```
theorem contender_13_lt_contender_14 : contender_13 < contender_14 := by
  simpa [contender_13, contender_14, family13] using
    (Nat.lt_trans (family13_lt_F14 42) (input_lt_F14 (F14 42)))
```

The proof has no numerical evaluation. Lean only checks the definitions, structural recursion, and inequalities.

10 Why this is a significant rather than cosmetic increase

A strict inequality alone can be cheap. The present construction contains several independently meaningful amplifiers.

Layer	Contender 13	Contender 14
Object language	dynamic-kind System F_ω	same certified language
Oracle interface	one unary family from Contender 12	packed countable family at every reflection node
Visible domination	current unary oracle value	every coordinate $fs(i,n)$ with $i \le n$
Reflection geometry	one direct reflection	recursive operator stages
Schedules	none in final wrapper	rank-dependent, countably diagonal schedules above `tree12`
Meta-rank	one reflection	$\mathsf{family13}(n)$ operator-stage levels
Formal margin	$C_{12} < C_{13}$	all lower ranks are pointwise below all higher ranks; bounded dynamic-kind programs are below F_{14}; and $C_{13} < C_{14}$

The rank theorems are particularly important. At input 42, the height is exactly C_{13}. A direct theorem exhibits every successor increase through a concrete branch, and a second theorem closes those steps transitively to compare arbitrary ranks. Together with the program-domination theorem, the construction contains an internally certified chain of strict upgrades indexed by the predecessor's full construction. No attempt is made to compress this into a speculative single ordinal label.

10.1 Why no exact fast-growing calibration is claimed

The repository's later contenders combine impredicative normalization, higher-kinded operators, a predicative dynamic universe, oracle reflection, and countably branching meta-schedules. Assigning a proof-theoretic ordinal to that entire combination would require a separate ordinal analysis, including a precise correspondence between the encoded object languages, the semantic normalization

arguments, and the external operator-stage recursion. The present paper therefore distinguishes three levels of assertion:

1. **Kernel-level intended theorem:** the supplied Lean source states the exact strict inequalities described above.
2. **Structural mathematical claim:** the new number iterates a progressive dynamic-family reflector through a predecessor-indexed tower, with strict successor embeddings.
3. **Ordinal heuristic:** deliberately left open pending a dedicated calibration theorem.

This is stronger scholarship than assigning an impressive but unsupported ordinal expression.

11 Computability and consistency discipline

11.1 Totality of the new definitions

Every new recursion is visibly structural.

- `reflect14` is finite arithmetic plus finite list maxima and the existing total `Fd.fvalue`.
- `schedule14` constructs one W-tree node and, on each branch, applies existing total syntax constructors and conversions.
- `diagonalOperatorTower` recurses structurally on a natural rank.
- `tower14`, `F14`, and `contender_14` are ordinary transparent definitions.

Evaluation is fantastically infeasible but computable in principle. This is exactly the contest's intended distinction.

11.2 No new normalization theorem is needed

A tempting route beyond Contender 13 would be an impredicative universe that contains itself, or kinds quantifying over kinds without a stratification. Such systems risk Girard-Hurkens-style inconsistency and would require a new strong normalization proof before their program maxima could be used constructively. Contender 14 avoids that risk. It reuses the already established `Fd.sn_of_typed` theorem and changes only the oracle interface and the external reflection geometry.

11.3 Expected axiom closure

The new arithmetic and transformer proofs themselves use ordinary constructive reasoning. Through the imported normalization development, the repository expects the permitted axioms `propext` and `Quot.sound`. The supplied audit additions request `#print axioms` for the new number and principal theorems and reject any forbidden predecessor constants in the definition closure.

12 No-laziness analysis

The rule against laziness is not merely textual. One could write a trivial successor of the old closed number and obtain a formal inequality without mathematical progress. Contender 14 differs in four ways.

1. The executable source of `contender_14` contains no reference to `contender_13` or any earlier closed contender constant.
2. The previous construction appears as a function `family13 : Nat -> Nat`, not as a single numeral.
3. That family is used as the oracle seed of a stronger reflected language and as the rank of an operator-stage tower.
4. The strict successor-rank theorem is independent of the old contender: it applies to every seed family and every rank.

The theorem `tower14_rank_at_42` is intentionally stated only in the proof module:

```
theorem tower14_rank_at_42 :
    tower14 42 =
      diagonalOperatorTower schedule14 fdFamilyTransformer contender_13 := by
  rfl
```

It documents the scale of the rank while keeping the closed predecessor constant out of the new definition's dependency closure.

13 Alternatives considered and rejected

13.1 A single second application of `F13`

Defining $F_{13}^{(k)}(42)$ for a small fixed k would be easy and formally larger, but it would not introduce a new mechanism. It would also sit close to the no-laziness boundary. The present proposal instead changes the oracle arity, reflection geometry, schedules, and meta-rank.

13.2 One operator stage over the dynamic reflector

A direct analogue of Contender 12 using the stronger dynamic reflector is a reasonable intermediate candidate. It is, however, dominated in design by the diagonal tower: rank zero recovers the base reflector, rank one already contains one operator stage, and every further rank is formally strictly larger.

13.3 Assuming oracle monotonicity

This would simplify many comparisons but would be mathematically wrong. The explicit prefix maximum and canonical leaf branch were included precisely to make every inequality direct.

13.4 A self-containing dynamic universe

This might appear to offer a more dramatic language-strength jump, but without a stratified model and normalization proof it would jeopardize the constructive basis of the program maximum. A serious future contender should first build a ranked universe hierarchy and prove its normalization, rather than silently crossing into an inconsistent calculus.

13.5 Using the predecessor constant as the tower height

The definition could have used `contender_13` directly. That would fail the repository's dependency audit and obscure whether the prior construction was genuinely integrated. The chosen `family13 n` formulation exposes the underlying function uniformly at every input and supports the pointwise theorem `family13_lt_F14`.

14 Integration and validation protocol

The archive contains a unified patch and an overlay. The intended integration steps are:

1. add `BusyLean/Contender14.lean`;
2. add `BusyLean/Comparison14.lean`;
3. import both from `BusyLean.lean`;
4. update `scripts/check.py` so Contender 13 is the unbudgeted previous phase and Contender 14 is the current definition phase;
5. extend `test/Audit.lean` with axiom prints, closure exclusions, and theorem checks;
6. run the static overlay validator;
7. run a clean `python3 scripts/check.py` under Lean 4.34.0;
8. run `lake env lean test/Audit.lean` independently and retain the full output.

14.1 What has been validated in this archive

The following checks were executed locally:

- all required source and theorem names are present;
- no forbidden construct occurs in executable source text;
- the new definition text does not mention any closed contender constant from Contender 5 through Contender 13;
- the final definition and predecessor-family equations match the intended construction;
- the unified patch is syntactically valid and applies to files with the recorded baseline contexts;
- the LaTeX article compiles, and the resulting PDF is rendered and visually inspected;
- all distributed files receive SHA-256 checksums.

14.2 What has not been validated here

The environment used to prepare this archive had neither a Lean 4.34.0 binary nor a materialized checkout of BusyLean. Consequently the following claims are not made:

- that the Lean elaborator accepted every source line;
- that the kernel checked every theorem;
- that `#print axioms` returned only the permitted axioms;
- that the definition and proof phases met the 15-second and 60-second budgets.

These are integration gates, not optional polish. The source was written to mirror existing theorem signatures and proof patterns closely, but the archive labels its status honestly as a serious, review-ready proposal rather than a falsely certified build.

15 Further research questions

15.1 Formal questions directly suggested by Contender 14

1. **Internal-node program domination.** The supplied `fvalue_lt_F14` theorem handles the canonical rank-zero reflection node. Index arbitrary reflection nodes inside nested `operatorStage` schedules and prove that every program value at every such node is below the root value $F_{14}(n)$.
2. **Quantitative rank-path certificates.** Refine `diagonalOperatorTower_rank_lt` so its proof returns an explicit branch path (or a list of local inequalities) witnessing how rank a embeds into rank b. This would make the large-margin certificate computationally inspectable rather than merely propositional.
3. **Schedule cofinality.** Define an explicit embedding relation on W-tree schedules and prove that the positive branches of `schedule14 k` are cofinal in the repository's finite `Autonomous.hyper` hierarchy above `tree12`.
4. **Definition-closure audit theorem.** Turn the current metaprogram audit into a reusable theorem or verified command proving that construction families, but no closed predecessor constants, occur in each contender's dependency closure.
5. **Compilation-cost control.** Measure whether the symbolic rank `family13 n` stays opaque enough during elaboration to meet the 15-second definition budget. If not, reorganize declarations to prevent accidental reduction while preserving transparency and executability.

15.2 Stronger language directions

6. **Ranked dynamic universes.** Replace one kind `dyn` by a predicative hierarchy $\mathrm{dyn}_0, \mathrm{dyn}_1, \ldots$, where level $r+1$ packages operators over kinds formed using lower levels. Prove strong normalization uniformly in r, add a sound checker, and reflect over a runtime-selected finite rank.
7. **Tree-valued object language.** Extend the strongest normalized language so certified programs can return W-trees to Lean, not only Church numerals. This could replace externally supplied schedules by schedules generated by the strongest language itself, echoing the self-indexing that made Contender 6 effective.

8. **Universe-polymorphic family coding.** Internalize the packed family interface at the type level, so object-language programs can quantify over family representations and certified decoders rather than relying on a fixed 2-adic pairing.

9. **Proof-relevant normalization bounds.** Extract a certified bound on normalization height from the reducibility proof and reflect over those bounds directly. This may expose information currently erased by the normal form evaluator.

15.3 Calibration and metamathematics

10. **Ordinal analysis of dynamic-kind F_ω.** Identify a theory whose provably total numerical functions match the accepted object language, including two-level kind variables and the predicative dynamic universe.

11. **External reflection versus internal provability.** Determine whether the family reflector and operator-stage tower correspond to iterated uniform reflection principles over that theory, and under what coding assumptions.

12. **Conservative lower bounds.** Embed standard fast-growing hierarchies or iterated inductive-definition systems into the object language to obtain rigorous lower bounds before attempting an exact calibration.

13. **Upper bounds from semantic ranks.** Assign well-founded ranks to the reducibility-candidate semantics and the external W-tree recursion, then prove a compositional upper bound for the complete construction.

16 Conclusion

Contender 14 is designed as the natural synthesis of the repository's two most powerful recent ideas: Contender 13's predicative universe of operators and Contender 12's operator-level reflection. The new family reflector exposes a countable oracle family to the strongest certified object language while retaining direct visible-coordinate bounds. A rank-dependent schedule then iterates the operator closure, and the predecessor family itself selects the number of ranks. The explicit leaf branch provides a reusable strict-rank witness; its finite transitive closure compares arbitrary ranks, and the rank-zero path lifts bounded dynamic-kind program values below F_{14}. Consequently the large-margin claim rests on structural inequalities rather than oracle monotonicity or speculative ordinal slogans.

The central equations are compact:

$$\mathsf{family13}(n) = F_{13}(F_{13}(n)), \qquad F_{14}(n) = T_{\mathsf{family13}(n)}(\mathsf{family13})(n,n), \qquad C_{14} = F_{14}(F_{14}(42)),$$

and the intended formal conclusion is

$$\boxed{C_{13} < C_{14}}.$$

The accompanying overlay, patch, audit additions, validation script, and full source listings make the proposal ready for kernel checking and repository review.

A Theorem dependency map

Declaration	Role and principal dependencies
`lt_reflect14`	Arithmetic fact for the outer $n+1+\max$ wrapper.
`oracle_lt_reflect14`	Uses `W.le_prefixMax` and `lt_reflect14`.
`fvalue_lt_reflect14`	Uses `W.le_maximum` and `lt_reflect14`.
`fdFamilyTransformer_progressive`	Immediate from visible-family domination.
`diagonalOperatorTower_progressive`	Natural-number induction plus the existing `operatorStage_progressive`.
`transformer_lt_operatorStage_schedule14`	Chains `transformer_lt_lift_at` with `operatorStage_child_lt` through branch zero.
`diagonalOperatorTower_step_lt`	Instantiates the preceding theorem at rank k using tower progressivity.
`diagonalOperatorTower_rank_lt`	Inductively closes strict successor steps to compare arbitrary finite ranks.
`family13_lt_F14`	Applies tower progressivity to the constant predecessor seed at visible coordinate zero.
`input_lt_F13`	Reuses `Fd.input_lt_reflectFd`.
`input_lt_family13`	Two transitive uses of `input_lt_F13`.
`input_lt_F14`	Chains input growth of the predecessor family with `family13_lt_F14`.
`lower_rank_lt_F14`	Instantiates the arbitrary-rank theorem below the self-indexed final rank.
`fvalue_lt_F14`	Composes base-program domination with the rank-zero-to-final path.
`tower14_rank_at_42`	Definitional equality showing the exact meta-rank.
`contender_13_lt_contender_14`	Pointwise predecessor domination followed by input growth.

B Lean definition module

```
import BusyLean.Contender13
import BusyLean.MetaReflection
import BusyLean.Autonomous

/-!
# Contender 14: self-indexed diagonal operator reflection

The reflected object language is Contender 13's dynamic-kind System F-omega,
but the unary oracle is now a packed countable family.  A separate prefix
maximum makes the resulting transformer progressive without assuming the
(false) monotonicity of reflection in its oracle.

The base transformer is then lifted through a diagonal tower of operator
stages.  At rank `k`, `schedule14 k` has a canonical leaf branch (used by the
strict comparison proofs) and countably many branches carrying successively
higher autonomous schedules above Contender 12's `tree12`.

The tower height at input `n` is `family13 n = F13 (F13 n)`.  In particular,
the height at input 42 is definitionally Contender 13, but the definition never
mentions the constant `contender_13` itself.
```

```
```
family13 n = F13 (F13 n)
tower14 n   = diagonalOperatorTower schedule14 fdFamilyTransformer (family13 n)
F14 n   = tower14 n (fun _ => family13) n n
contender_14 = F14 (F14 42)
```

All definitions are transparent, structural, and executable.  Do not evaluate
`contender_14`.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- Dynamic-kind System F-omega reflection with a countable oracle family.

The family is presented to the object language through `packFamily`.  The
explicit prefix maximum is logically separate: it ensures that every visible
seed coordinate `fs i n`, for `i <= n`, is below the reflector even though
object-language evaluation is not monotone in the oracle. -/
def reflect14 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (Fd.fvalue (packFamily fs) n) (Fd.ctmsUpTo n))

/-- One family-aware dynamic-kind reflection step, packaged as a transformer.
The output coordinate is ignored, exactly as for the earlier primitive
reflection transformers. -/
def fdFamilyTransformer : Transformer := fun fs _ n => reflect14 fs n

/-- A rank-dependent autonomous schedule.

Branch zero is a leaf, giving a canonical embedded copy of one autonomous lift.
Branch `r+1` carries the `(k+r+1)`-st higher autonomous derivative above
Contender 12's schedule.  Thus every rank exposes an unbounded diagonal family
of schedules while retaining an explicit branch for direct inequalities. -/
def schedule14 (k : Nat) : WTree Nat :=
  .node (fun j =>
    match j with
    | 0 => .leaf
    | r + 1 =>
        Autonomous.realize
          (Autonomous.hyper (k + r + 1) (Autonomous.encode tree12)))

/-- Iterate operator-stage closure along a rank-dependent schedule.

At successor rank, the entire transformer from the previous rank becomes the
base transformer for a fresh operator stage. -/
def diagonalOperatorTower (schedule : Nat → WTree Nat)
    (base : Transformer) : Nat → Transformer
  | 0 => base
  | k + 1 =>
      operatorStage (diagonalOperatorTower schedule base k) (schedule k)
```

```
/-- Contender 13's construction as a function of its input.
Definitionally, `family13 42 = contender_13`. -/
def family13 (n : Nat) : Nat := F13 (F13 n)

/-- The self-indexed tower: its rank at input `n` is `family13 n`. -/
def tower14 (n : Nat) : Transformer :=
  diagonalOperatorTower schedule14 fdFamilyTransformer (family13 n)

/-- The diagonal operator tower, seeded by Contender 13's family. -/
def F14 (n : Nat) : Nat := tower14 n (fun _ => family13) n n

/-- **Contender 14.** Never evaluate it. -/
def contender_14 : Nat := F14 (F14 42)

end BusyLean
```

C Lean comparison module

```
import BusyLean.Contender14
import BusyLean.MetaReflectionFacts
import BusyLean.Comparison13

/-! # Contender 14: comparison theorems -/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- The arithmetic core of family-aware dynamic-kind reflection. -/
theorem lt_reflect14 {fs : W.Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
      (BigNumber.maximum (Fd.fvalue (packFamily fs) n) (Fd.ctmsUpTo n))) :
    m < reflect14 fs n := by
  show m < n + 1 + _
  have hM : ∀ M : Nat, M + 1 ≤ n + 1 + M := fun M => by
    rw [Nat.add_comm M 1]
    exact Nat.add_le_add_right (Nat.le_add_left 1 n) M
  exact Nat.lt_of_le_of_lt h
    (Nat.lt_of_lt_of_le (Nat.lt_succ_self _) (hM _))

/-- Every visible coordinate of the oracle family is strictly exceeded. -/
theorem oracle_lt_reflect14 (fs : W.Family) {i n : Nat} (hi : i ≤ n) :
    fs i n < reflect14 fs n :=
  lt_reflect14
    (Nat.le_trans (W.le_prefixMax (fun j => fs j n) hi)
      (BigNumber.le_maxN_left _ _))

/-- Every enumerated dynamic-kind program, run with the packed family oracle,
is strictly exceeded by one reflection step. -/
theorem fvalue_lt_reflect14 (fs : W.Family) {n : Nat} {t : Fd.CTm}
```

```
    (ht : t ∈ Fd.ctmsUpTo n) :
    Fd.fvalue (packFamily fs) n t < reflect14 fs n :=
  lt_reflect14
    (Nat.le_trans (W.le_maximum (Fd.fvalue (packFamily fs) n) ht)
      (BigNumber.le_maxN_right _ _))

/-- The numerical input is strictly exceeded by one reflection step. -/
theorem input_lt_reflect14 (fs : W.Family) (n : Nat) : n < reflect14 fs n :=
  Nat.lt_of_lt_of_le (Nat.lt_succ_self n) (Nat.le_add_right _ _)

/-- Family-aware dynamic-kind reflection is a progressive transformer. -/
theorem fdFamilyTransformer_progressive : Progressive fdFamilyTransformer := by
  intro seed i o n hi
  exact oracle_lt_reflect14 seed hi

/-- Every finite rank of the diagonal operator tower is progressive. -/
theorem diagonalOperatorTower_progressive
    (schedule : Nat → WTree Nat) (base : Transformer)
    (hbase : Progressive base) (k : Nat) :
    Progressive (diagonalOperatorTower schedule base k) := by
  induction k with
  | zero => simpa [diagonalOperatorTower] using hbase
  | succ k ih =>
      simpa [diagonalOperatorTower] using
        (operatorStage_progressive
          (diagonalOperatorTower schedule base k) ih (schedule k))

/-- One use of `schedule14 k` strictly raises every visible value of a
progressive transformer.  The proof uses only the explicit leaf at branch zero;
it does not assume any monotonicity of reflection in an oracle. -/
theorem transformer_lt_operatorStage_schedule14 {base : Transformer}
    (hbase : Progressive base) (seed : W.Family) (k : Nat)
    {i o n : Nat} (hi : i ≤ n) :
    base seed i n < operatorStage base (schedule14 k) seed o n := by
  let f : Nat → WTree Nat := fun j =>
    match j with
    | 0 => .leaf
    | r + 1 =>
        Autonomous.realize
          (Autonomous.hyper (k + r + 1) (Autonomous.encode tree12))
  have h1 : base seed i n < liftTransformer base seed 0 n :=
    transformer_lt_lift_at hbase seed hi
  have h2 : operatorStage base (f 0) seed 0 n <
      operatorStage base (.node f) seed o n :=
    operatorStage_child_lt base hbase seed f (Nat.zero_le n)
  have h2' : liftTransformer base seed 0 n <
      operatorStage base (schedule14 k) seed o n := by
    simpa [f, schedule14] using h2
  exact Nat.lt_trans h1 h2'

/-- Consecutive ranks of the concrete tower are pointwise strictly increasing
at every visible coordinate. -/
theorem diagonalOperatorTower_step_lt (seed : W.Family) (k : Nat)
    {i o n : Nat} (hi : i ≤ n) :
```

```
    diagonalOperatorTower schedule14 fdFamilyTransformer k seed i n <
      diagonalOperatorTower schedule14 fdFamilyTransformer (k + 1) seed o n := by
  simpa [diagonalOperatorTower] using
    (transformer_lt_operatorStage_schedule14
      (diagonalOperatorTower_progressive schedule14 fdFamilyTransformer
        fdFamilyTransformer_progressive k)
      seed k hi)

/-- Every lower rank is pointwise strictly below every higher rank at visible
coordinates.  This is the finite transitive closure of the one-step theorem. -/
theorem diagonalOperatorTower_rank_lt (seed : W.Family)
    {k l i o n : Nat} (hi : i ≤ n) (hkl : k < l) :
    diagonalOperatorTower schedule14 fdFamilyTransformer k seed i n <
      diagonalOperatorTower schedule14 fdFamilyTransformer l seed o n := by
  induction l generalizing k i o with
  | zero =>
      exact False.elim (Nat.not_lt_zero k hkl)
  | succ l ih =>
      cases Nat.lt_or_eq_of_le (Nat.le_of_lt_succ hkl) with
      | inl hlt =>
          exact Nat.lt_trans
            (ih (k := k) (i := i) (o := 0) hi hlt)
            (diagonalOperatorTower_step_lt seed l (i := 0)
              (o := o) (n := n) (Nat.zero_le n))
      | inr heq =>
          subst k
          exact diagonalOperatorTower_step_lt seed l hi

/-- At every input, `F14` strictly exceeds Contender 13's entire family. -/
theorem family13_lt_F14 (n : Nat) : family13 n < F14 n := by
  change family13 n <
    diagonalOperatorTower schedule14 fdFamilyTransformer (family13 n)
      (fun _ => family13) n n
  exact
    (diagonalOperatorTower_progressive schedule14 fdFamilyTransformer
      fdFamilyTransformer_progressive (family13 n))
      (fun _ => family13) 0 n n (Nat.zero_le n)

/-- `F13` is input-increasing. -/
theorem input_lt_F13 (n : Nat) : n < F13 n :=
  Fd.input_lt_reflectFd family12 n

/-- The family used as the rank and seed of Contender 14 is input-increasing. -/
theorem input_lt_family13 (n : Nat) : n < family13 n :=
  Nat.lt_trans (input_lt_F13 n) (input_lt_F13 (F13 n))

/-- `F14` is input-increasing. -/
theorem input_lt_F14 (n : Nat) : n < F14 n :=
  Nat.lt_trans (input_lt_family13 n) (family13_lt_F14 n)

/-- The self-indexing rank is positive at every input. -/
theorem zero_lt_family13 (n : Nat) : 0 < family13 n :=
  Nat.lt_of_le_of_lt (Nat.zero_le n) (input_lt_family13 n)
```

```
/-- Every proper lower tower rank is below the selected self-indexed rank. -/
theorem lower_rank_lt_F14 {k n : Nat} (hk : k < family13 n) :
    diagonalOperatorTower schedule14 fdFamilyTransformer k
        (fun _ => family13) 0 n < F14 n := by
  change diagonalOperatorTower schedule14 fdFamilyTransformer k
      (fun _ => family13) 0 n <
    diagonalOperatorTower schedule14 fdFamilyTransformer (family13 n)
      (fun _ => family13) n n
  exact diagonalOperatorTower_rank_lt (fun _ => family13)
    (i := 0) (o := n) (n := n) (Nat.zero_le n) hk

/-- Even the rank-zero family-aware dynamic reflector is below `F14`. -/
theorem reflect14_seed_lt_F14 (n : Nat) :
    reflect14 (fun _ => family13) n < F14 n := by
  simpa [diagonalOperatorTower, fdFamilyTransformer] using
    (lower_rank_lt_F14 (k := 0) (n := n) (zero_lt_family13 n))

/-- Every dynamic-kind program enumerated at depth `n`, run with the packed
Contender-13 family oracle, is strictly below `F14 n`. -/
theorem fvalue_lt_F14 {n : Nat} {t : Fd.CTm} (ht : t ∈ Fd.ctmsUpTo n) :
    Fd.fvalue (packFamily (fun _ => family13)) n t < F14 n :=
  Nat.lt_trans (fvalue_lt_reflect14 (fun _ => family13) ht)
    (reflect14_seed_lt_F14 n)

/-- The tower rank at input 42 is definitionally the previous contender. -/
theorem tower14_rank_at_42 :
    tower14 42 =
      diagonalOperatorTower schedule14 fdFamilyTransformer contender_13 := by
  rfl

/-- **The required comparison:** Contender 13 is strictly smaller than
Contender 14. -/
theorem contender_13_lt_contender_14 : contender_13 < contender_14 := by
  simpa [contender_13, contender_14, family13] using
    (Nat.lt_trans (family13_lt_F14 42) (input_lt_F14 (F14 42)))

/-- All previously accepted contenders are therefore smaller as well. -/
theorem contender_12_lt_contender_14 : contender_12 < contender_14 :=
  Nat.lt_trans contender_12_lt_contender_13 contender_13_lt_contender_14

end BusyLean
```